



OppMon

Arkon Data Model

Complete entity-relationship reference for the Arkon multi-tenant PostgreSQL schema — 82 tables across 11 logical domains, including TimescaleDB hypertables and pgvector embedding stores.

Document version	1.0
Last updated	2026-05-07 · post migration ID-cutover
Database	PostgreSQL 15
Extensions	TimescaleDB · pgvector · pgcrypto · uuid-ossdp
Schema source	Prisma + raw-SQL migrations
Total tables	82 (+ _migrations runner state)
Domains	11 logical domains
Primary key style	TEXT cuid via gen_random_uuid()::text

Generated from canonical schema documentation

Contents

Contents	2
Overview	4
Domain Map	5
1. Identity & Tenancy	6
2. Agent Runtime & Events	8
3. LLM Sessions, Routing & MCP	9
4. Memory Subsystem	10
5. RAG (Retrieval-Augmented Generation)	11
6. Journal & Work Tracking	12
7. Workflows, Skills & Tasks	13
8. Audit, Usage & Pricing	14
9. Notifications	15
10. Infrastructure, Cost & Tracing	16

11. Security & Rate Limit	17
Full Table Inventory	18
Migration Consolidation — Error → Fix Lookup	20
Hypertable internal IDs	20
PostgreSQL Extensions & Key Indexes	21
Extensions	21
Key Indexes	21
Operational Quick-Reference	23

Overview

Hybrid schema management

The OppMon (Arkon) database is a hybrid Prisma-managed plus raw-SQL schema running on PostgreSQL 15. Prisma owns the core multi-tenant tables (Tenant, User, Team, Agent, etc.) defined in `packages/database/prisma/schema.prisma` and applied via `prisma db push`. Raw SQL migrations under `apps/api/scripts/migrations/` add features Prisma cannot model: pgvector columns, TimescaleDB hypertables, RLS policies, triggers, full-text search, and the journal/memory subsystems. The migration runner is `apps/api/scripts/migrate.ts` — it sorts SQL files alphabetically and tracks state in `_migrations`.

Identifier conventions

All IDs are TEXT/cuid. Legacy SERIAL/INTEGER ids were converted to TEXT PRIMARY KEY DEFAULT `gen_random_uuid()::text` during the 2026-05-09 cutover. The exception is the three TimescaleDB hypertable internal counters (`node_metrics`, `traces`, `spans`) which keep BIGSERIAL because they have no FK targets.

Column naming

Snake_case in Postgres; Prisma fields are camelCase mapped via `@map()`.

Special tenants

The schema is seeded with three system tenants: `default` (seeded in `001_create_tenants.sql`), `transformate` (seeded in `017_journal.sql`), and `system` (seeded in `2026-05-08_rls_and_rbac.sql` for SYSTEM_ADMIN operations).

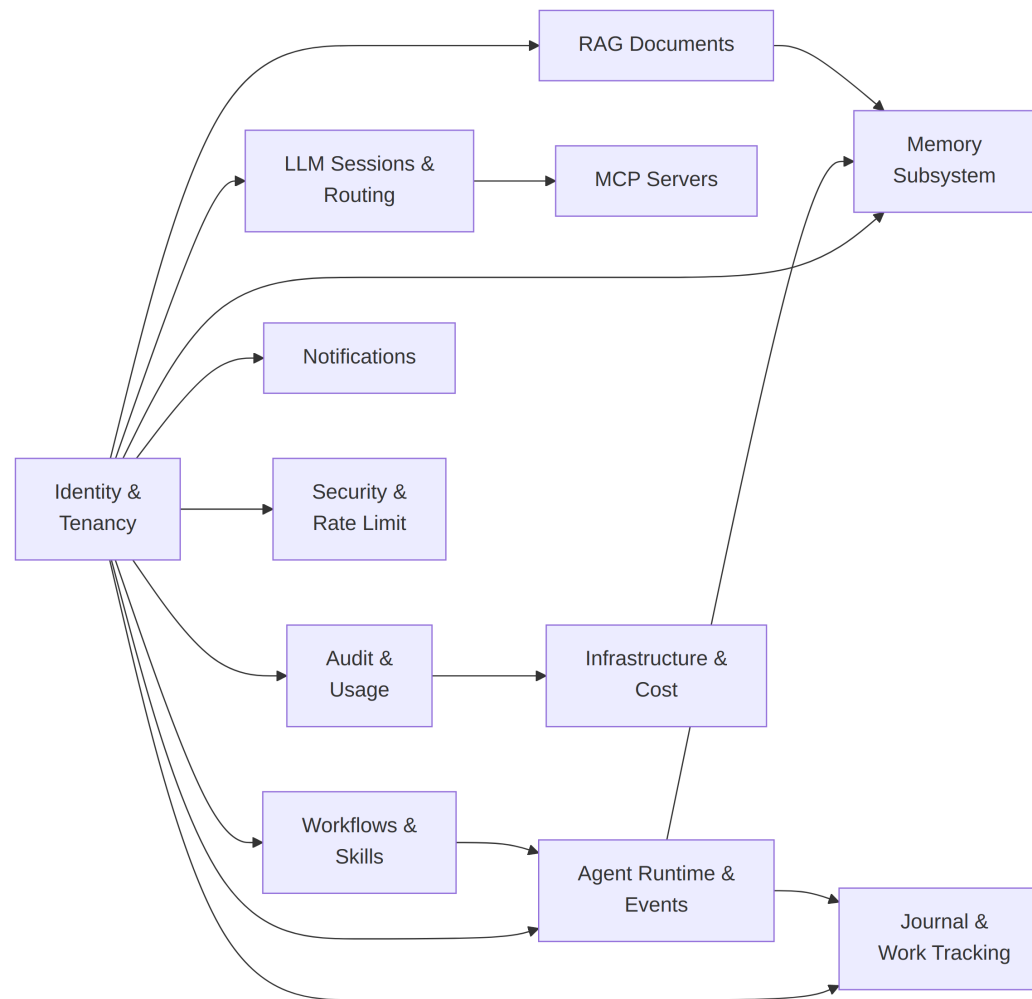
How to read this document

First the cross-domain map shows how the 11 domains relate. Then each domain gets its own ERD with table-level detail — column names, types, and the most important enum values. Finally the appendix contains the full 82-table inventory, the migration error cookbook, the extension list, and the key indexes that hold the schema together at runtime.

Domain Map

High-level relationships between the 11 logical domains

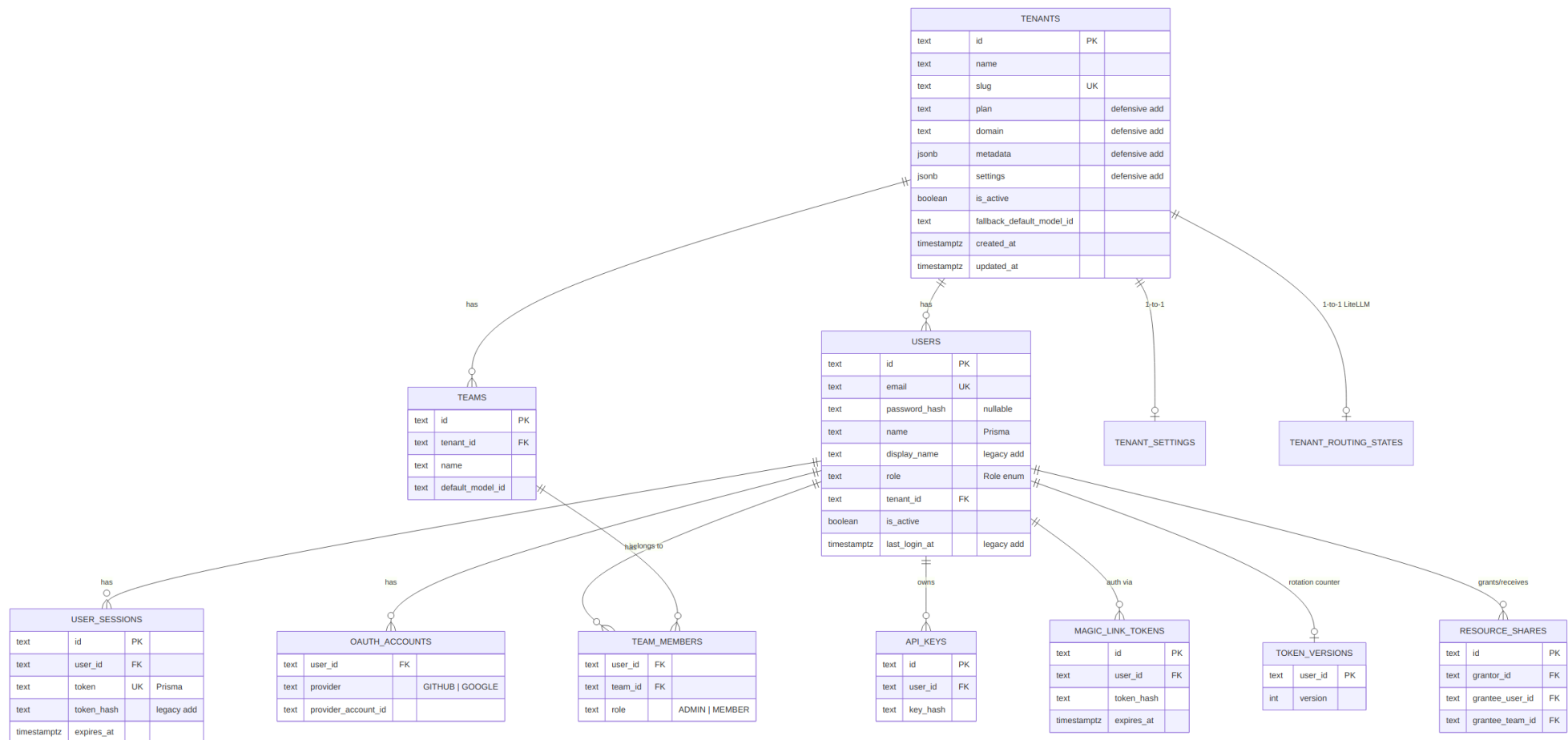
Identity & Tenancy is the root: every tenant-scoped table FKs to tenants(id) with ON DELETE CASCADE. Agent Runtime feeds Memory and the Journal; LLM Sessions reach into MCP for tool exposure; RAG flows into Memory; Workflows orchestrate Agents; and Audit feeds Infrastructure cost analysis.



1. Identity & Tenancy

Root of the multi-tenant model — users, teams, sessions, OAuth, API keys, magic links

Every tenant-scoped relation cascades from `tenants(id)`. Users belong to exactly one tenant via `users.tenant_id`, and Many-to-Many membership of teams flows through `team_members`. The `token_versions` table is a per-user counter that invalidates JWTs on rotation. `resource_shares` grants scoped access to specific resources across users and teams.



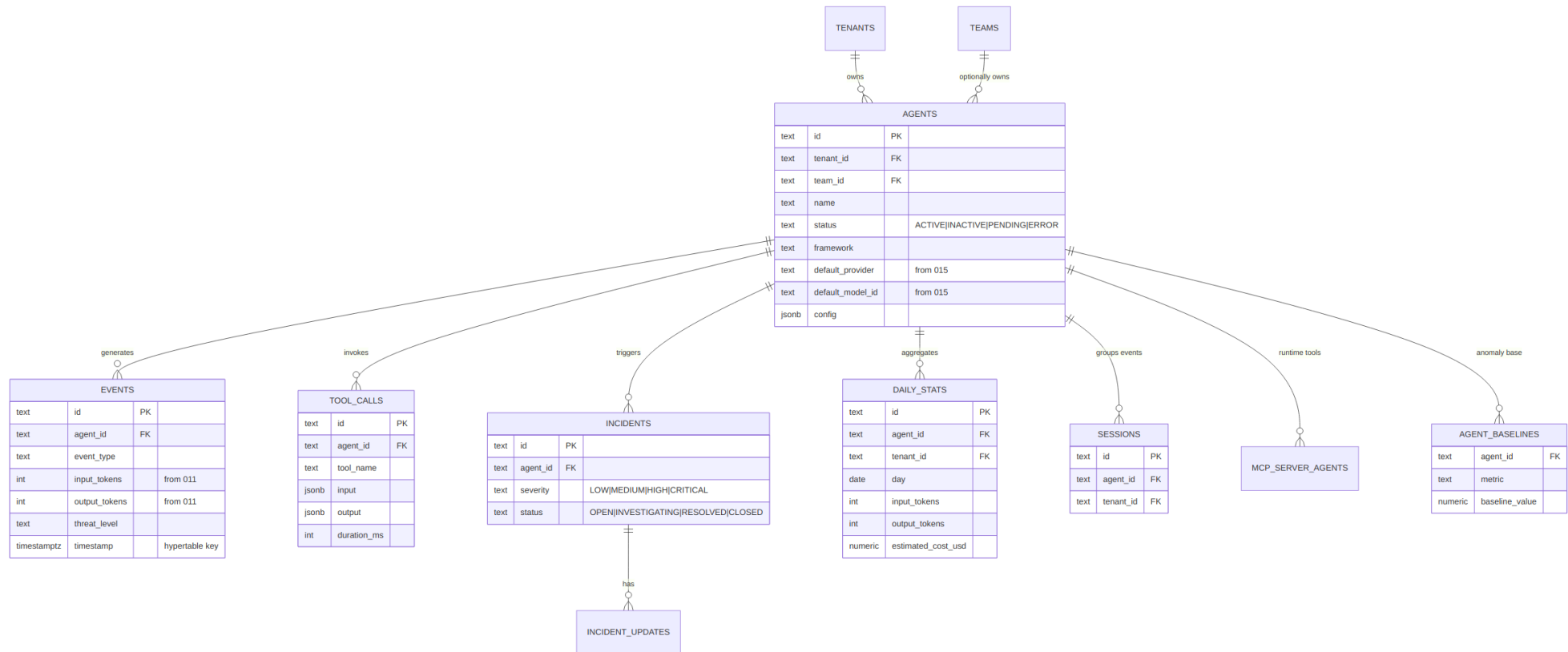
Defensive add columns: tenants gained plan, domain, metadata, settings; users gained display_name and last_login_at; user_sessions added token_hash alongside the Prisma token UK column.

Special seeded tenants: default, transformate, system.

2. Agent Runtime & Events

Live agent telemetry — TimescaleDB hypertable for events, daily aggregates, incidents

events is a TimescaleDB hypertable keyed on timestamp; daily aggregates land in daily_stats. Tool invocations are logged in tool_calls with input/output JSON for replay. Anomaly detection compares live metrics to agent_base_lines; deviations open incidents with severity and status enums.

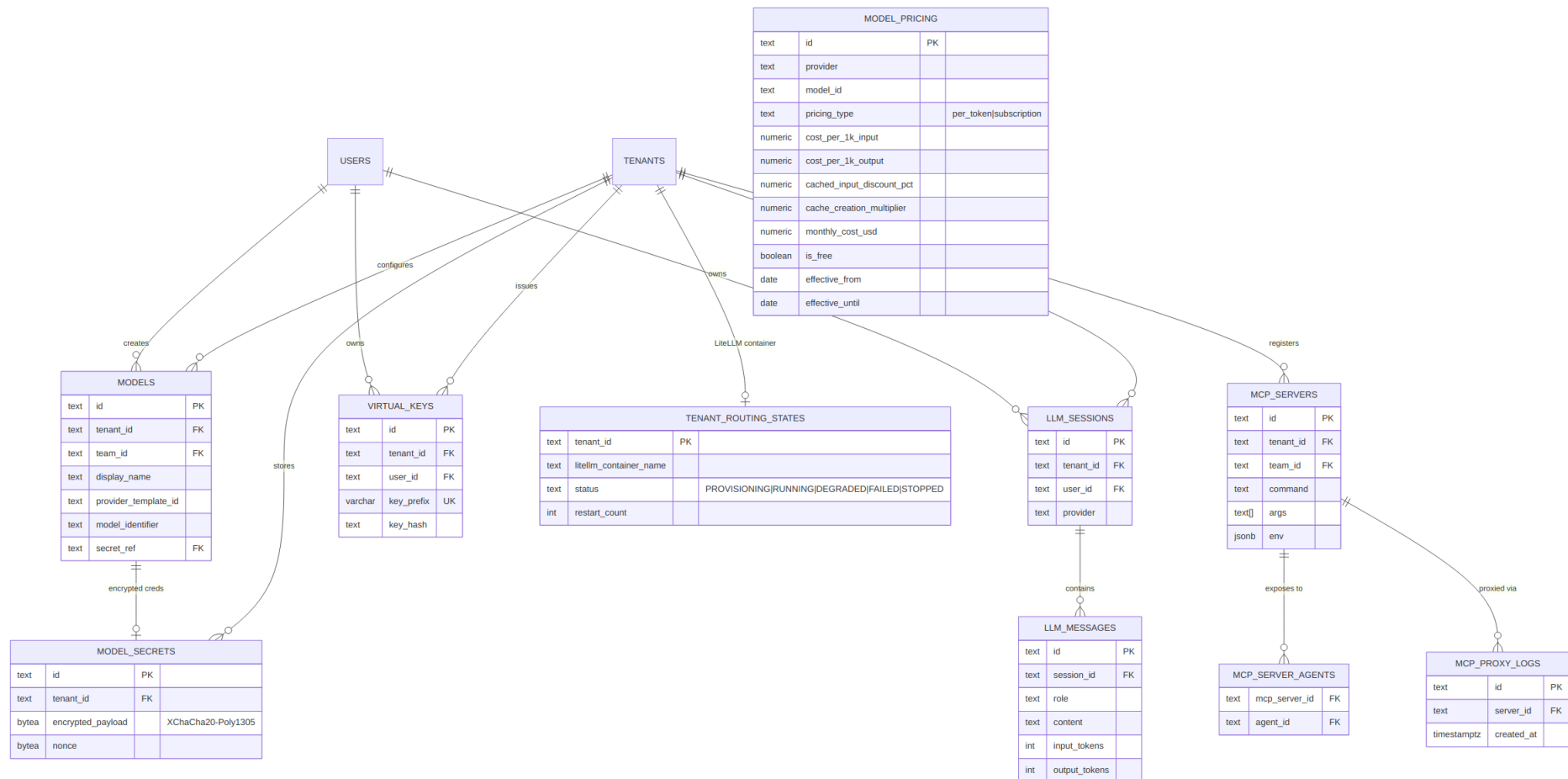


Migrations 011 (token columns on events) and 015 (default_provider/default_model_id on agents) are the most-touched migrations in this domain.

3. LLM Sessions, Routing & MCP

Tenant-scoped LLM chat plus the LiteLLM-backed routing config and MCP server registry

`tenant_routing_states` tracks the per-tenant LiteLLM container lifecycle (PROVISIONING → RUNNING → DEGRADED / FAILED / STOPPED). `model_secrets` stores XChaCha20-Poly1305 encrypted credentials with a separate nonce column. Virtual keys give tenants short `key_prefix` + `key_hash` lookup that doesn't leak the full key. MCP servers expose tools to specific agents via the `mcp_server_agents` M2M, and every proxied request is logged in `mcp_proxy_logs`.

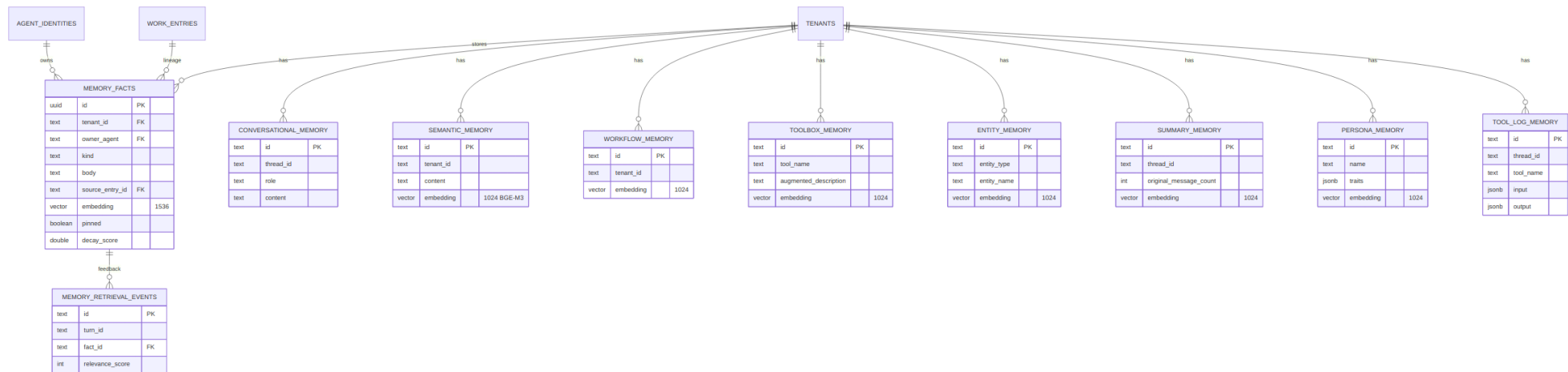


model_pricing is a global rate-card table (no tenant_id). Seeded with 63 rows: Anthropic 11, OpenAI 21, Cerebras 10, Ollama 21 by 2026-05-09_seed_model_pricing.sql.

4. Memory Subsystem

Two parallel memory designs coexist — the v2 fact store and the 8-table memory bank

memory_facts (018_memory_v2.sql) is the v2 fact store: pgvector 1536-dim MRL-truncated embeddings, per-(tenant, agent) scope, with HNSW or IVFFlat indexes depending on pgvector version. The **8-table memory bank** (024_prisma_schema_alignment.sql) uses 1024-dim BGE-M3 embeddings, with one table per memory type: conversational, semantic, workflow, toolbox, entity, summary, persona, and tool_log. Prisma cannot model vector (N) columns — embedding columns are added defensively right before the HNSW index creation block.

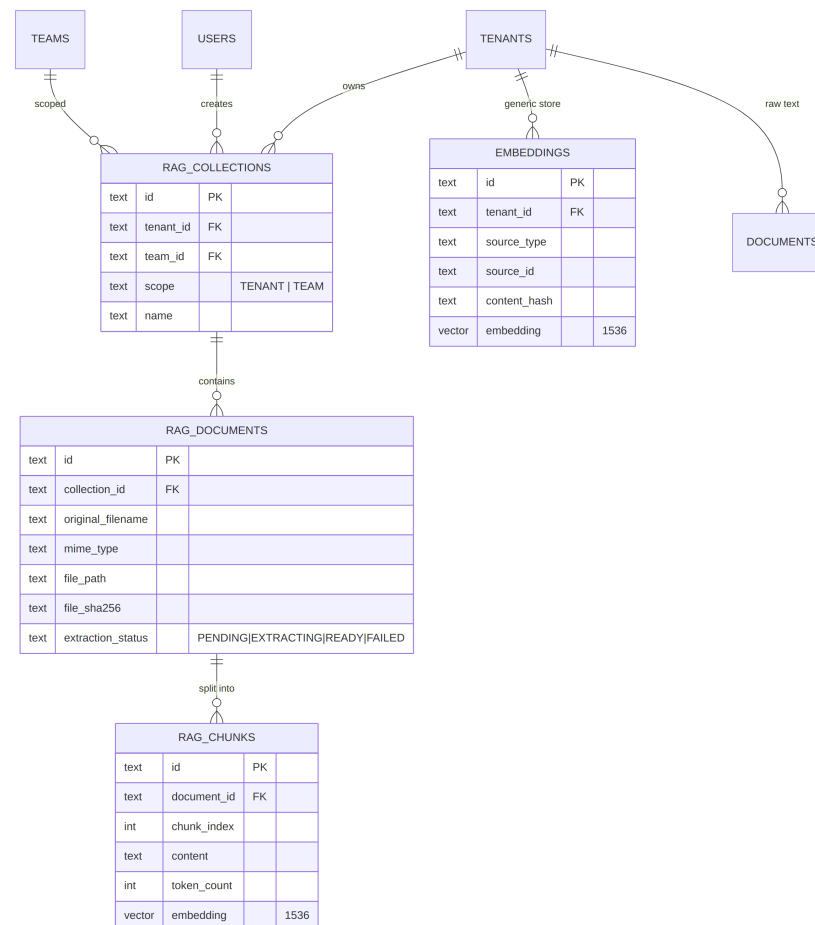


memory_retrieval_events feeds back relevance scores, enabling decay and reinforcement of facts over time.

5. RAG (Retrieval-Augmented Generation)

Tenant- and team-scoped collections, documents, and 1536-dim chunk embeddings

RAG collections can be scoped TENANT or TEAM. Documents track extraction lifecycle (PENDING → EXTRACTING → READY / FAILED) and a SHA-256 of the source file for dedup. Chunks carry token_count for cost estimation and a 1536-dim pgvector embedding indexed by HNSW. The generic embeddings table is a separate vector store keyed by (source_type, source_id) for non-RAG use cases. documents is an unrelated raw-text store.

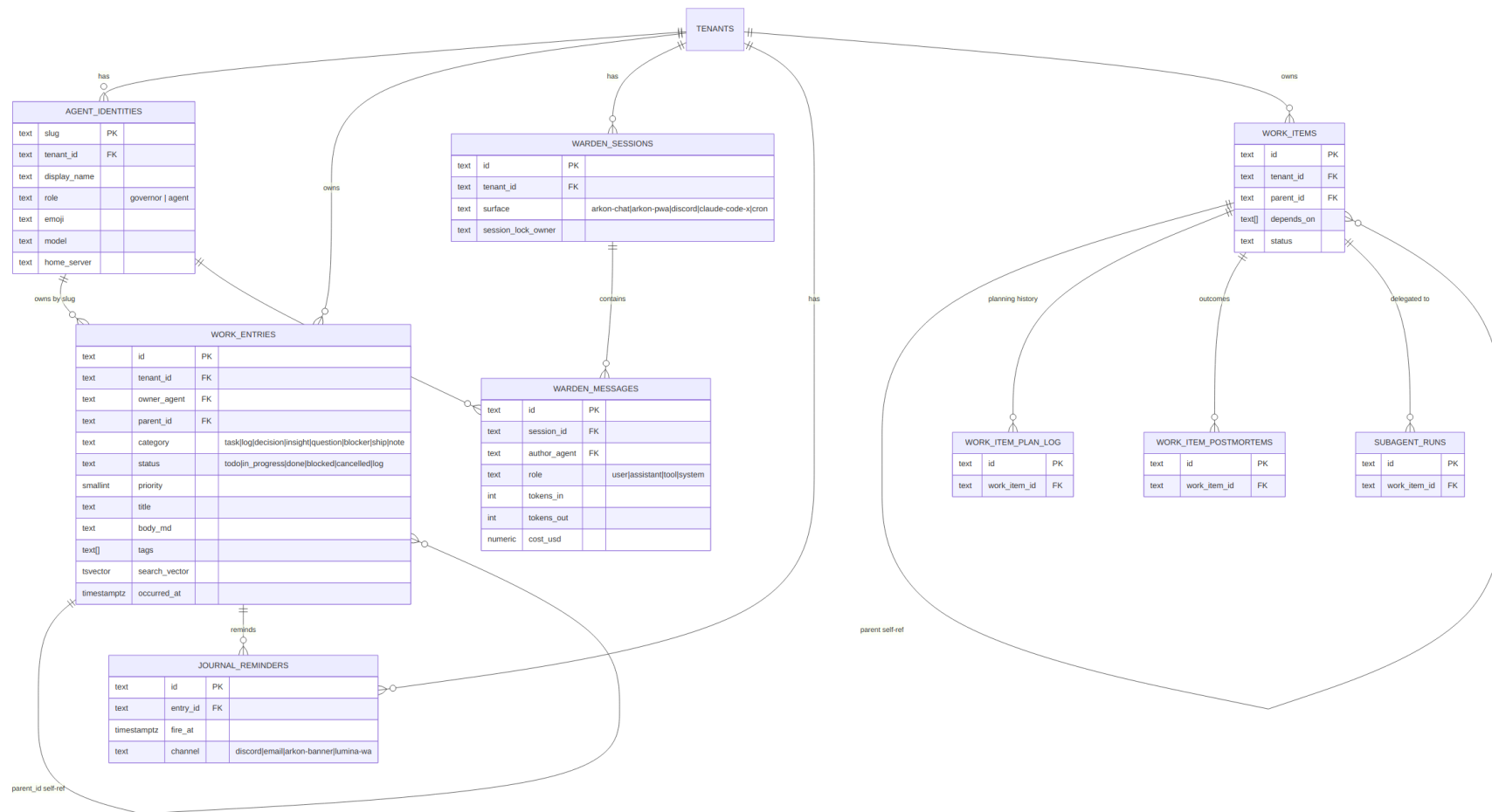


Both `rag_chunks.embedding` and `embeddings.embedding` are added defensively in `024_prisma_schema_alignment.sql` — Prisma cannot model `vector(N)` columns.

6. Journal & Work Tracking

The Warden + governed-agent unified work tracker plus the work-items system

Two related but distinct systems live here. **Journal** (017_journal.sql) tracks work_entries by category — task / log / decision / insight / question / blocker / ship / note — with status, priority, tags, full-text search_vector, and self-referential parent/child links. **Work-items** (020_work_items.sql) is the heavier ticket system, with planning history, postmortems, and delegated subagent runs. The warden_* tables provide conversational continuity across surfaces (arkon-chat, PWA, Discord, claude-code, cron).

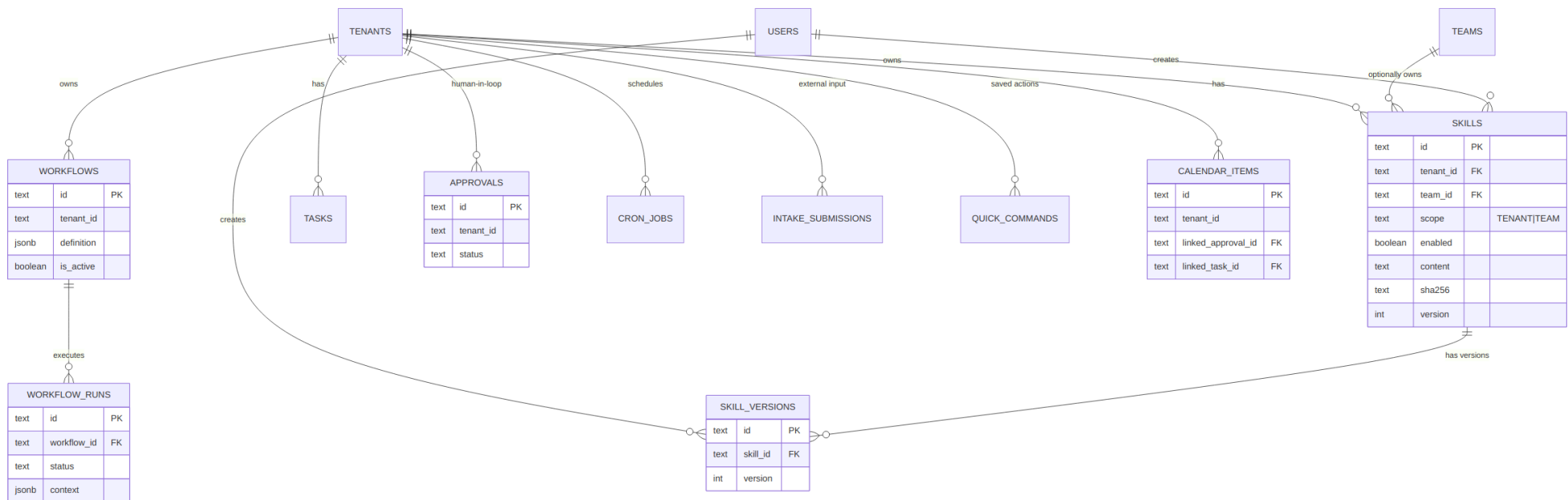


agent_identities is seeded with 8 agents for the transformate tenant: warden, brynn, lumina, sentinel, scout, codesmith, hermes, opus-desktop.

7. Workflows, Skills & Tasks

Definition + execution surfaces for automation, plus skills, approvals, and calendar

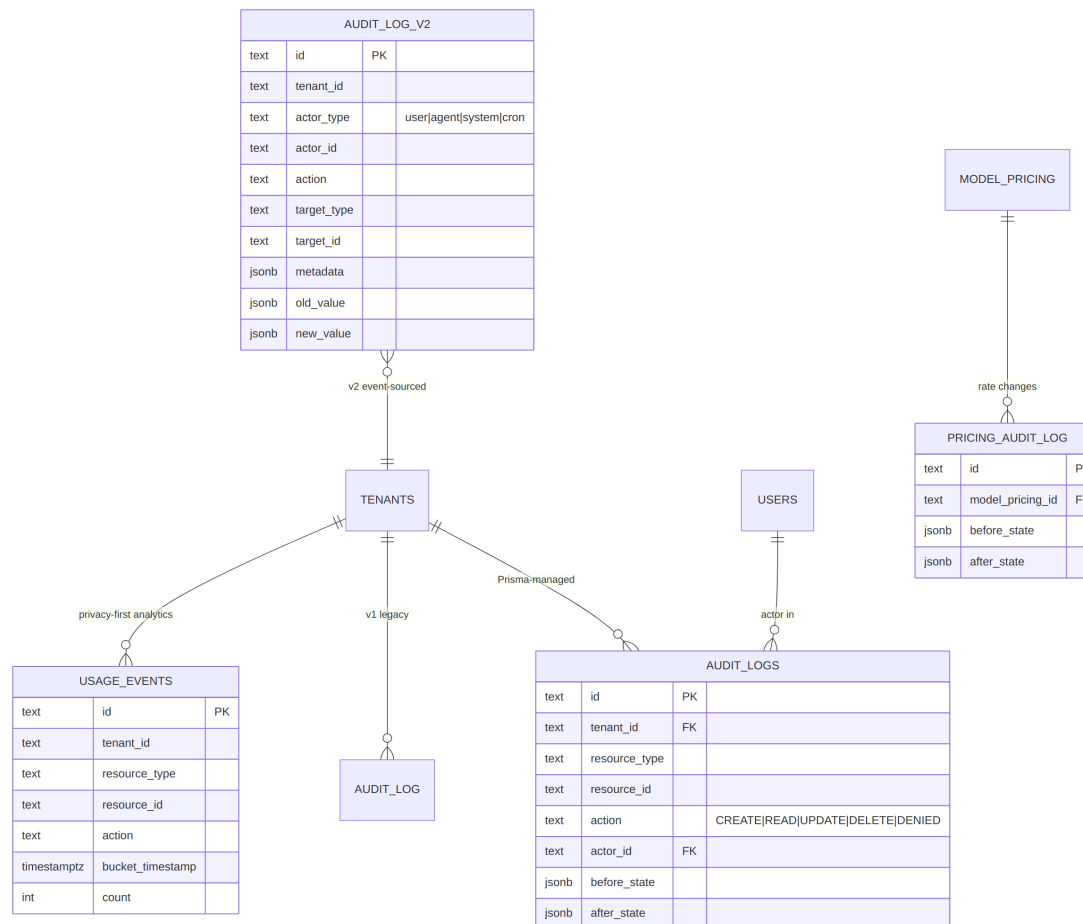
workflows hold versioned JSONB definitions; runs are recorded in workflow_runs with status + execution context. skills are versioned text blobs (with SHA-256 fingerprint) scoped TENANT or TEAM, and skill_versions retains every revision. approvals back human-in-loop gates; calendar_items can link to either an approval or a task. cron_jobs, intake_submissions, and quick_commands round out the automation surface.



8. Audit, Usage & Pricing

Three coexisting audit/usage tables for different fidelity and retention levels

`audit_logs` (Prisma-managed) records actor + before/after JSONB per resource action — CREATE, READ, UPDATE, DELETE, DENIED. `audit_log_v2` (007) is the append-only event-sourced version with `actor_type` in (user, agent, system, cron). UPDATE and DELETE are revoked from the `mcadmin` role in production, and a BEFORE INSERT trigger enforces `tenant_id` integrity. `usage_events` is the privacy-first analytics path — no PII, just bucketed counters.

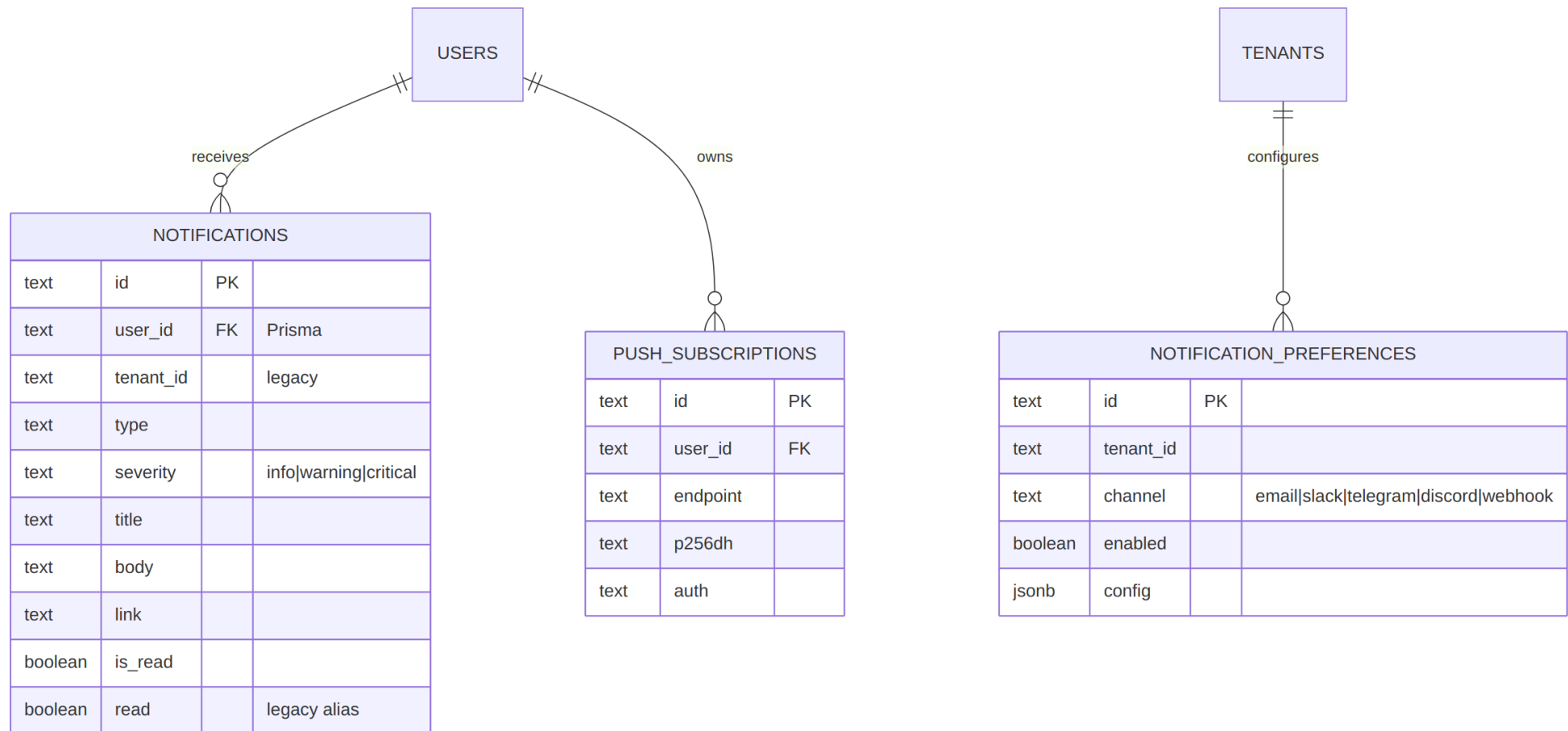


model_pricing changes are tracked in pricing_audit_log with full before/after JSONB so historical cost calculations remain reproducible.

9. Notifications

Two parallel notification stacks share one table — Prisma per-user + legacy per-tenant

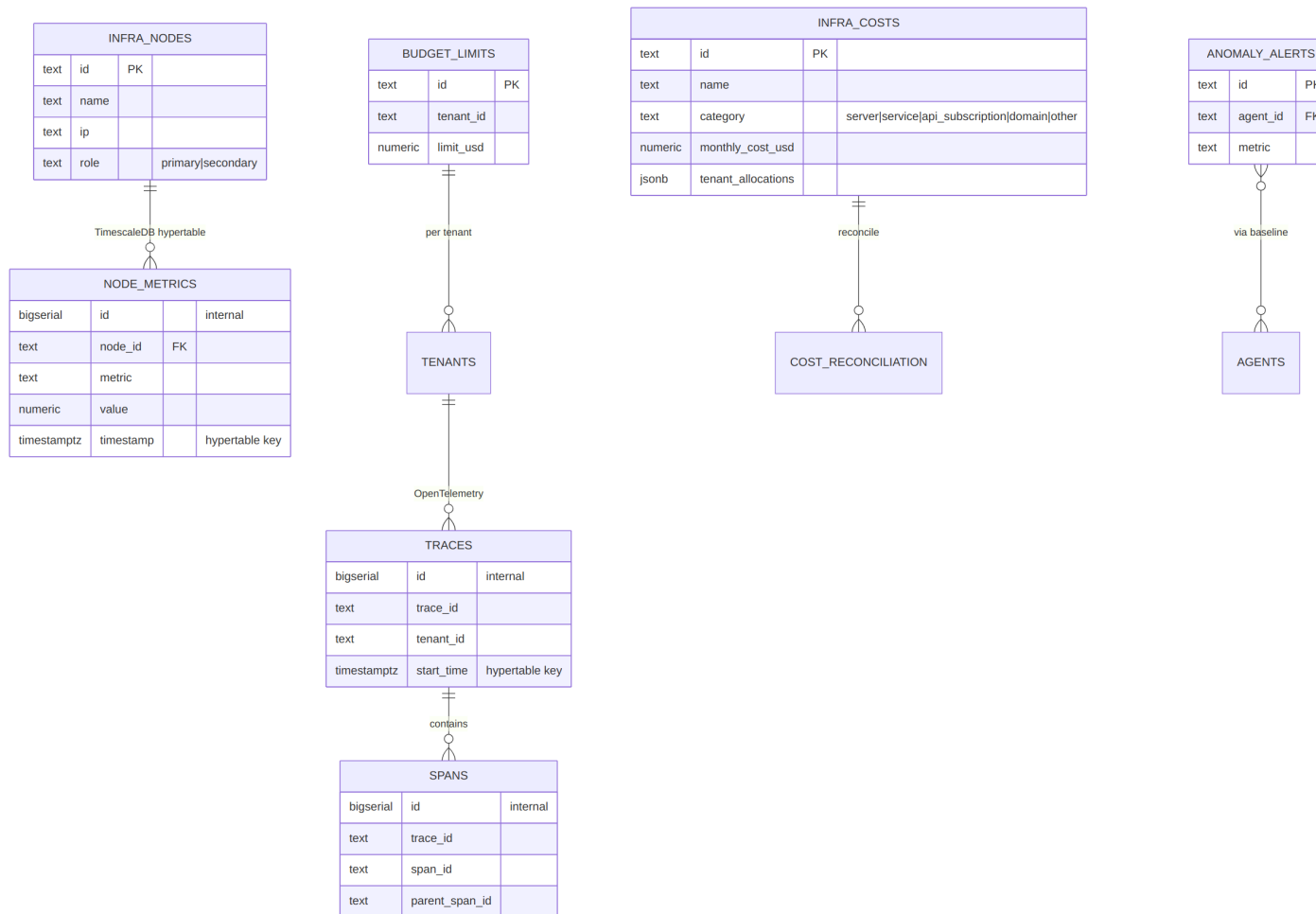
The notifications table reconciles two designs by carrying both shapes of columns: the Prisma per-user shape (`user_id`, `is_read`) and the legacy per-tenant shape (`tenant_id`, `severity`, `body`, `link`, `read`). Channel configuration lives in `notification_preferences`; Web Push subscriptions live in `push_subscriptions` with `endpoint`, `p256dh`, and `auth`.



10. Infrastructure, Cost & Tracing

Node telemetry, OpenTelemetry traces/spans, infrastructure cost reconciliation

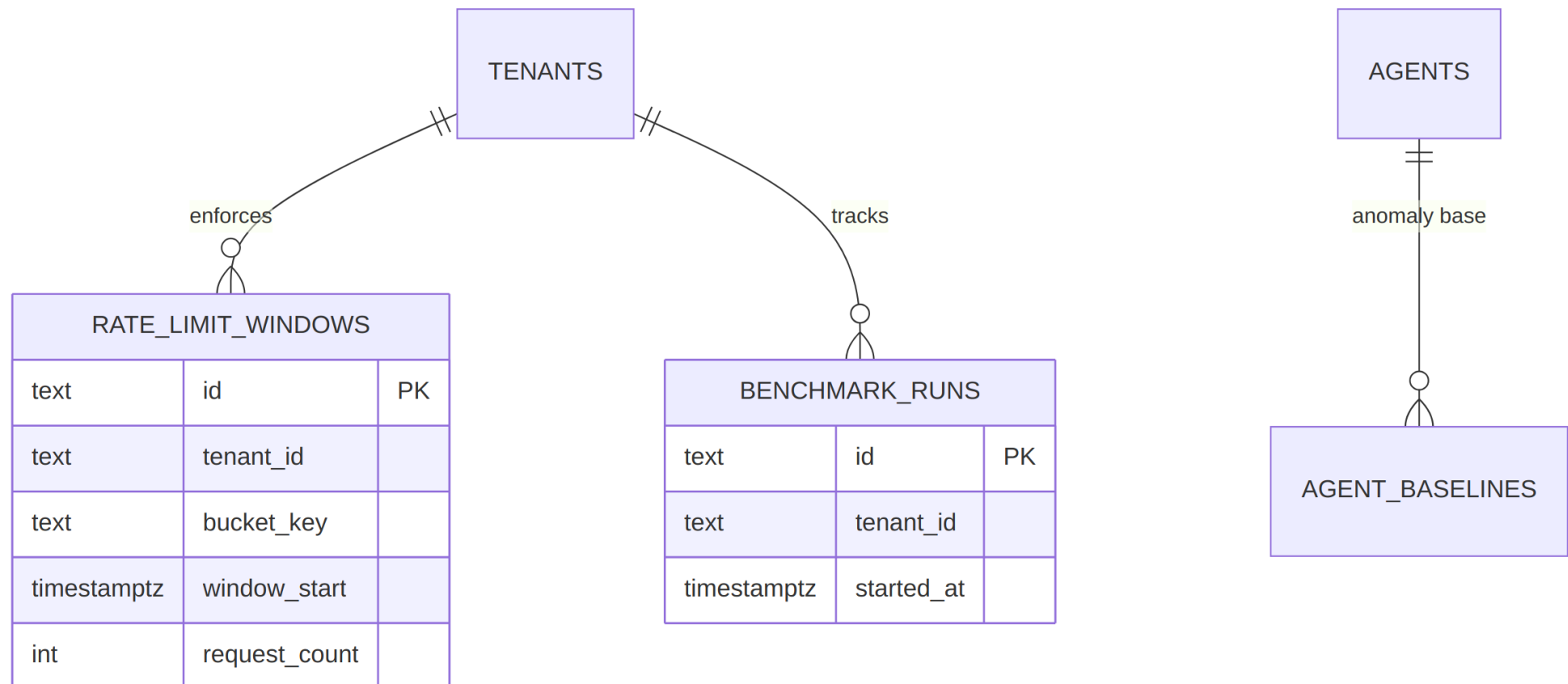
node_metrics, traces, and spans are TimescaleDB hypertables — they keep BIGSERIAL internal id counters because they have no FK targets. infra_costs tracks fixed monthly spend per server / service / API subscription / domain, with per-tenant allocation in JSONB. cost_reconciliation matches those allocations against vendor invoices. Per-tenant spend caps live in budget_limits. Anomaly alerts are generated by agents.



11. Security & Rate Limit

Token-bucket rate limit windows, benchmarks, and the RLS / RBAC perimeter

`rate_limit_windows` implements token-bucket state per (`tenant_id`, `bucket_key`) with a sliding `window_start`. The full security posture also includes RLS policies on every tenant-scoped table (2026-05-08_rls_and_rbac.sql), the `oppmon_app` DB role created with `NOSUPERUSER NOBYPASSRLS`, and a `BEFORE INSERT` `audit_logs` trigger for defense-in-depth.



Full Table Inventory

All 82 tables across the 11 domains. The *Source* column shows where the table is created — Prisma schema, a numbered raw-SQL migration, the legacy `base_schema.sql`, or the migration runner itself. `youtube_channels` was created in 016 and dropped in 021 and is therefore not counted.

#	Table	Source	Purpose	#	Table	Source	Purpose
1	tenants	Prisma + 001	Multi-tenant root	42	toolbox_memory	024	Tool description memory (1024)
2	users	Prisma + 006	Operator accounts	43	entity_memory	024	Entity extraction memory (1024)
3	user_sessions	Prisma + 006	JWT/session store	44	summary_memory	024	Conversation summaries (1024)
4	oauth_accounts	Prisma	GitHub/Google OAuth	45	persona_memory	024	Per-persona memory (1024)
5	teams	Prisma	Sub-tenant grouping	46	tool_log_memory	024	Tool exec audit (memory layer)
6	team_members	Prisma	User↔Team M2M	47	work_entries	017	Unified work tracker
7	tenant_settings	Prisma	Per-tenant flags	48	warden_sessions	017	Warden conv. continuity
8	tenant_routing_states	Prisma	LiteLLM container status	49	warden_messages	017	Warden session messages
9	token_versions	Prisma multitenant_redesign	JWT rotation counter	50	journal_reminders	017	Reminder scheduler
10	resource_shares	Prisma multitenant_redesign	Cross-user resource grants	51	work_items	020	Work-item tickets
11	api_keys	010	API key auth	52	work_item_plan_log	020	Plan revision history
12	magic_link_tokens	010	Passwordless email auth	53	work_item_postmortems	020	Outcome write-ups
13	agents	Prisma + 015	Registered agents	54	subagent_runs	020	Delegated subagent runs
14	events	Prisma + 011	Time-series events (hypertable)	55	tasks	base_schema	Generic tasks
15	tool_calls	024	Per-tool invocations	56	approvals	base_schema	Human-in-loop approvals
16	daily_stats	011 + 024	Daily aggregates	57	cron_jobs	base_schema	Scheduled jobs
17	sessions	001	Agent conversation sessions	58	intake_submissions	base_schema	External form intake
18	incidents	Prisma	Incident records	59	quick_commands	base_schema	Saved command palette
19	incident_updates	Prisma	Incident comments	60	calendar_items	base_schema	Calendar entries
20	agent_baselines	base_schema	Anomaly baselines	61	workflows	Prisma	Workflow definitions
21	agent_identities	017	Warden agent registry	62	workflow_runs	Prisma	Workflow exec records
22	mcp_servers	Prisma + 024	MCP server registry	63	skills	Prisma	Skill registry
23	mcp_server_agents	base_schema	MCP↔Agent M2M	64	skill_versions	Prisma	Skill version history
24	mcp_proxy_logs	base_schema	MCP request log	65	audit_logs	Prisma + 2026-05-08	Resource audit log
25	models	Prisma	Per-tenant model config	66	audit_log	base_schema	Legacy v1 audit
26	model_secrets	Prisma	Encrypted provider creds	67	audit_log_v2	007	Append-only event-sourced audit

#	Table	Source	Purpose	#	Table	Source	Purpose
27	model_pricing	base + 015 + 022 + 2026-05-09	Global rate cards	68	usage_events	Prisma	Privacy-first usage analytics
28	pricing_audit_log	base_schema	model_pricing change log	69	notifications	Prisma + 004	User+tenant notifications
29	virtual_keys	Prisma	API gateway keys	70	notification_preferences	004	Per-channel notification config
30	llm_sessions	Prisma	LLM chat sessions	71	push_subscriptions	005	Web push subscriptions
31	llm_messages	Prisma	LLM chat messages	72	infra_nodes	base_schema	Server/node registry
32	embeddings	Prisma + 024	Generic vector store (1536)	73	node_metrics	base_schema	Node telemetry (hypertable)
33	documents	base_schema	Raw doc storage	74	infra_costs	013	Infrastructure cost tracker
34	rag_collections	024	RAG collection grouping	75	cost_reconciliation	014	Cost vs invoice reconciliation
35	rag_documents	024	RAG-indexed documents	76	budget_limits	base_schema	Per-tenant spend caps
36	rag_chunks	024 + 003	RAG chunked content (1536)	77	traces	009	OTel traces (hypertable)
37	memory_facts	018	Long-term agent memory (1536, MRL)	78	spans	009	OTel spans (hypertable)
38	memory_retrieval_events	023	Memory relevance feedback	79	anomaly_alerts	base_schema	Anomaly alert records
39	conversational_memory	024	Per-thread chat memory	80	rate_limit_windows	008	Token-bucket rate limit state
40	semantic_memory	024	Tenant semantic memory (1024)	81	benchmark_runs	base_schema	Performance benchmarks
41	workflow_memory	024	Workflow exec memory (1024)	82	_migrations	migrate.ts	Migration runner state

Migration Consolidation — Error → Fix Lookup

The 2026-05-09 migration cutover converted every SERIAL/INTEGER id to TEXT/gen_random_uuid()::text and made every legacy migration idempotent against a Prisma db push-managed live database. When `pnpm --filter @oppmon/api migrate` fails, match the error to one of these patterns.

Error shape	Root cause	Fix
column "X" does not exist	Prisma's db push made a partial table; raw SQL CREATE TABLE IF NOT EXISTS no-op'd	Add ALTER TABLE Y ADD COLUMN IF NOT EXISTS X after the CREATE block
null value in column "X"	Prisma column has no DB default but is NOT NULL	Add the column to the INSERT explicitly (e.g. NOW() for updated_at, explicit slug)
duplicate key value violates unique constraint	ON CONFLICT (id) DO NOTHING only catches PK conflicts; another column has unique conflict	Switch to INSERT ... SELECT ... WHERE NOT EXISTS (... WHERE id = '...' OR slug = '...')
incompatible types: integer and text on FK	Legacy migration declared INTEGER REFERENCES but Prisma made the parent's id TEXT	Convert to TEXT REFERENCES
relation "X" already exists	Bare CREATE TABLE X	Switch to CREATE TABLE IF NOT EXISTS X
role "X" does not exist	Hardcoded prod-only DB role in GRANT/REVOKE	Wrap in DO \$\$ IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'X') THEN EXECUTE '...'; END IF; END \$\$
_id_seq does not exist	Sequence GRANT/trigger references a SERIAL that's now TEXT	Drop the sequence-related statement
violates foreign key constraint ..._tenant_id_fkey	Seed references a tenant that isn't seeded yet	Defensively INSERT ... SELECT ... WHERE NOT EXISTS for the missing tenant
syntax error at or near "\"	psql meta-command (\set, \if) in a .sql file	File is meant for psql CLI, not the migration runner — rename to .psql.skip
column "embedding" does not exist	Prisma can't model vector(N); column was never created	ALTER TABLE Y ADD COLUMN IF NOT EXISTS embedding vector(N) before the HNSW index

Hypertable internal IDs

node_metrics, traces, and spans deliberately keep id BIGSERIAL — they are TimescaleDB hypertable internal counters with no FK targets, so they don't need to be cuid'd.

PostgreSQL Extensions & Key Indexes

Extensions

Extension	Purpose	Version
TimescaleDB	Time-series optimization for events, node_metrics, traces, spans, work_entries (optional)	2.x
pgvector	Vector embeddings on 9+ tables (1024-dim BGE-M3 + 1536-dim OpenAI/Gemini/MRL)	≥0.5.0 (HNSW); falls back to IVFFlat if older
pgcrypto	gen_random_uuid() for cuid-style TEXT primary keys	built-in
uuid-oss	Legacy UUID generation (kept for compatibility)	built-in

Key Indexes

Table	Index	Columns
events	idx_events_agent_created	(agent_id, created_at DESC)
events	idx_events_type_timestamp	(event_type, timestamp DESC)
agents	idx_agents_tenant	(tenant_id)
agents	idx_agents_default_model	(default_provider, default_model_id)
daily_stats	idx_daily_stats_tenant	(tenant_id)
sessions	idx_sessions_tenant	(tenant_id)
users	idx_users_email	(email)
users	idx_users_tenant	(tenant_id)
notifications	idx_notifications_tenant_read	(tenant_id, read, created_at DESC)
memory_facts	idx_memory_facts_embedding_hnsw	HNSW(embedding vector_cosine_ops)

Table	Index	Columns
memory_facts	idx_memory_facts_body_fts	GIN(to_tsvector('english', body))
memory_facts	idx_memory_facts_pinned	(tenant_id, owner_agent) WHERE pinned
rag_chunks	idx_rag_chunks_vector	HNSW(embedding)
embeddings	idx_embeddings_vector	HNSW(embedding)
work_entries	idx_work_entries_search	GIN(search_vector)
work_entries	idx_work_entries_tags	GIN(tags)
audit_logs	idx_audit_logs_tenant_type_created	(tenant_id, resource_type, created_at DESC)
usage_events	idx_usage_events_tenant_bucket	(tenant_id, bucket_timestamp)
virtual_keys	idx_virtual_keys_prefix	(key_prefix)
model_pricing	idx_model_pricing_provider_model	(provider, model_id)

Operational Quick-Reference

Run migrations

```
pnpm --filter @oppmon/api migrate
```

Verify model_pricing seed (expected: anthropic=11, cerebras=10, ollama=21, openai=21)

```
SELECT provider, COUNT(*) FROM model_pricing GROUP BY provider;
```

Inspect a column's existence (used in defensive blocks)

```
SELECT 1 FROM information_schema.columns
WHERE table_schema = current_schema()
AND table_name = '<table>'
AND column_name = '<column>';
```

Inspect a role's existence

```
SELECT 1 FROM pg_roles WHERE rolname = '<role>';
```

Re-run a single migration (force) — edit _migrations to remove the row, then:

```
pnpm --filter @oppmon/api migrate
```

Stamp a migration as already-applied (escape hatch)

```
INSERT INTO _migrations (name) VALUES ('<filename without .sql>')
ON CONFLICT (name) DO NOTHING;
```

See apps/api/scripts/stamp_legacy_migrations.sql for the bulk version of the stamp escape hatch.